

32-bit MIPS Single Cycle Verification

**Amir Jamshidi Bandari
Electrical Engineering
Toronto Metropolitan University
March 2026**

SINGLE CYCLE VERIFICATION SUMMARY:

This document presents the verification process for the designed **32-bit MIPS single-cycle processor**. The processor was verified by writing instructions into instruction memory, creating testbenches, analyzing waveforms, and checking the produced signals.

Testing was performed for multiple instruction formats, including: R-type, I-type, J-type.

The verification was completed in a hierarchical order:

PC Flow: PC Adder, Branch Adder, Jump Concatenation, PC Register.

Arithmetic Units: ALU, Multiplier, Divider.

Register and Memory: HI/LO Register, Register File, Data Memory.

Control Logic: Instruction Decoder, Control Unit.

Writeback Path: Register Writeback 1.

The table below summarizes the hierarchical verification process:

Module	Verify	Cause	Reverify
PC Adder	Clear	-	-
Branch Adder	Error	Missing Instantiation	Clear
Jump Concatenation	Clear	-	-
PC Register	Clear	-	-
ALU	Error	Missing Instantiation	Clear
Multiplier	Clear	-	-
Divider	Clear	-	-
HILO Register	Clear	-	-
Register File	2 Errors	Missing Instantiation Illegal Overwrite	Clear
Data Memory	Clear	-	-
Instruction Decoder	Clear	-	-
Control Unit	Clear	-	-
Register_Writeback_1	Clear	-	-

Overall, four issues were found during verification. Three of them were caused by missing instantiations in the CPU top module, MIPS.sv, where module inputs or outputs were not properly connected. One issue was caused by an illegal overwrite, where the register file allowed register \$0 to be overwritten.

In the MIPS architecture, register \$0 must always remain equal to zero. This rule was originally broken, but the issue was fixed and reverified.

Every module that showed an error was debugged using waveform analysis and root-cause analysis. After the fixes, all affected modules were reverified successfully.

Note: Most verification tests require setup instructions before the target behavior occurs. Because of this, some screenshots may show extra clock cycles before the tested signal behavior appears.

1. PC Flow

- PC Adder

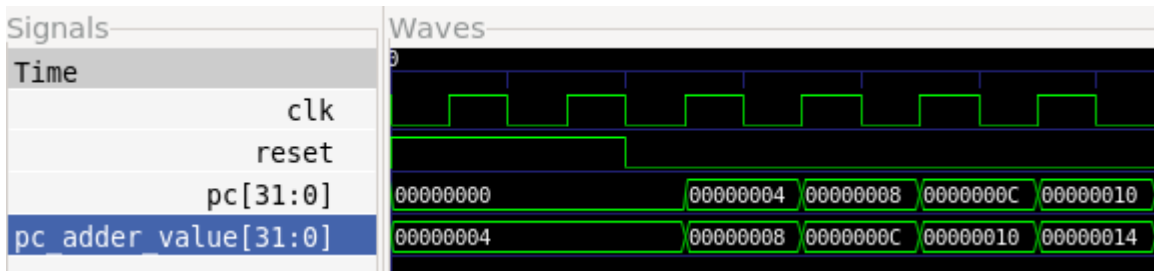


Image 1

Test Objective:

Verify that the PC adder increments the program counter address by **4** every clock cycle.

Signals Observed:

- clk
- pc_adder_value (hex)

Expected Behavior:

clk	pc_adder_value
Next rising edge	00000004
Next rising edge	00000008
Next rising edge	0000000C

Observed Behavior:

At each rising clock edge, the program counter increments by **4**, allowing the CPU to point to the next instruction in instruction memory. The program counter can also be redirected by control-flow instructions, such as a jump instruction.

Conclusion:

The PC adder correctly increments the program counter address by **4** each cycle. No issues were observed in this hardware block, and the unit works as expected.

- Branch Adder

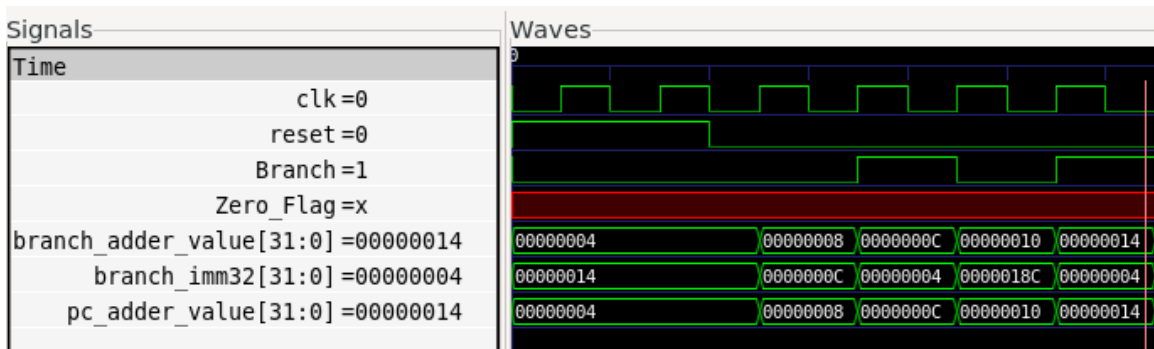


Image 2

Test Objective:

Verify that the branch adder calculates the correct branch target address when a branch instruction is taken.

Signals Observed:

- Branch
- Zero_Flag
- branch_adder_value (hex)
- branch_imm32 (hex)
- pc_adder_value (hex)

Expected Behavior:

Branch	Zero_Flag	pc_adder_value	branch_imm32	branch_adder_value
1	1	00000014	00000004	00000018

Observed Behavior:

Comparing the expected behavior with Image 2 shows that Zero_Flag and branch_adder_value are not behaving as expected. The Zero_Flag signal is outputting x, which means the value is undefined.

The expected branch target is 00000018, but the CPU continues to 00000014. The pc_adder_value behaves correctly, and branch_imm32 also behaves correctly because the instruction memory was set to skip one instruction, represented by branch_imm32 = 00000004.

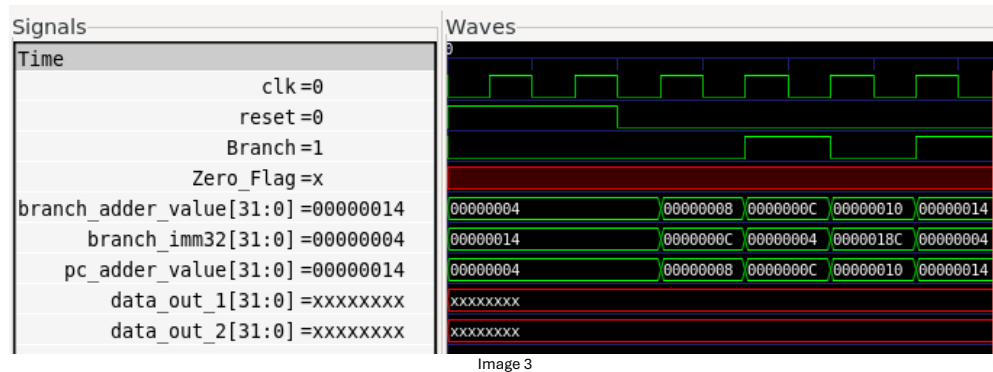
Root Cause:

Because Zero_Flag is undefined, the branch condition is not properly met, so the CPU continues executing the next instruction instead of

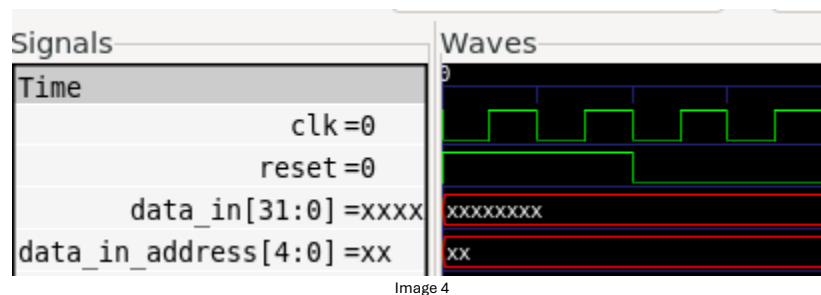
```
branch_adder_value = pc_adder_value;
```

branching. This behavior is caused by the following fallback assignment in the Branch_Adder module:

The main issue is the Zero_Flag. This flag must be either high or low, and it is generated by the ALU by comparing two register values.



Looking at Image 3, data_out_1 and data_out_2, which are the two register values being compared, are both undefined. Since the ALU is comparing undefined values, the Zero_Flag also becomes undefined. This bug is not caused by the ALU itself because the register output values should be initialized to 0 after reset.



In Image 4, the register writeback MUX also has undefined values and addresses, while the ALU output behaves correctly. This suggests that the issue is likely connected to the Register_Writeback_1 module.

Debugging Operation:

While debugging the Register_Writeback_1 module, the Register1_Destination signal did not appear in the GTKWave signal list. This indicated that the Register_Writeback_1 module was not properly connected to the CPU.

After checking the CPU top module, MIPS.sv, the writeback module was found to be missing its instantiation. To fix this issue, the Register_Writeback_1 module was instantiated in the CPU top module.

```
// REGISTER WRITEBACK
Register_Writeback_1 reg1_writeback (
    .ALU_result(ALU_result),
    .HI_result(HI_result),
    .LO_result(LO_result),
    .LUI_imm32_result(LUI_imm32_result),
    .rd(rd),
    .rt(rt),
    .Writeback_Control(Writeback_Control),
    .Register1_Destination(Register1_Destination),
    .load_word(load_word),
    .data_in(data_in),
    .data_in_address(data_in_address)
);
```

New observed Behavior:

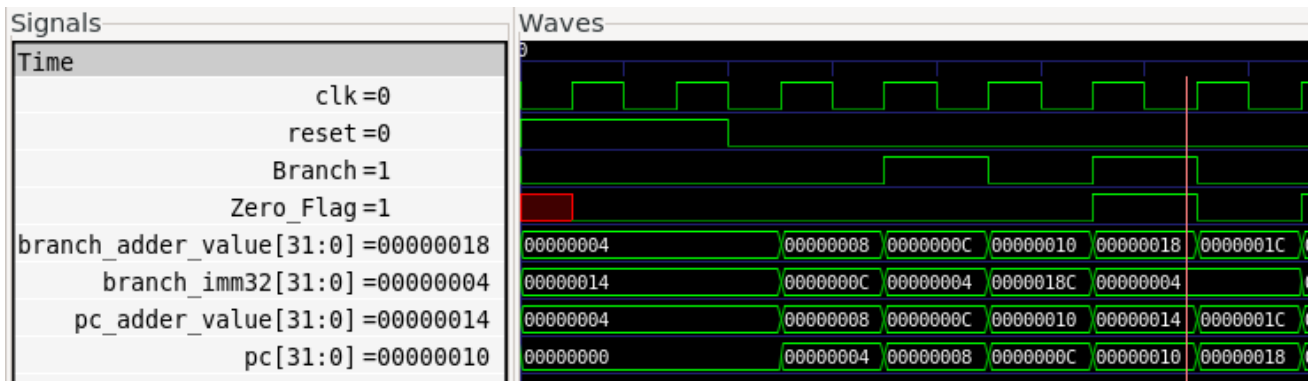


Image 5

After the fix, when both Branch and Zero_Flag are high, the branch target is correctly calculated as: $pc_adder_value + branch_imm32$

The program counter branches from address 00000010 to 00000018, which matches the expected behavior.

Conclusion:

The first observation showed a bug through the undefined Zero_Flag signal. The root cause was a missing module instantiation in the CPU top module, meaning part of the required hardware was not connected.

After instantiating the missing Register_Writeback_1 module, the branch adder behaved correctly. No further bugs were observed in this hardware block.

- Jump Concatenation

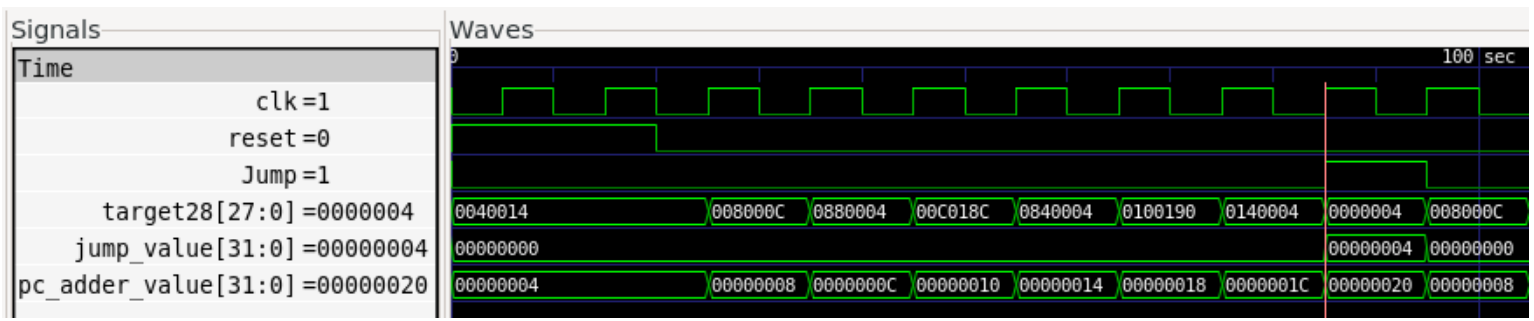


Image 6

Test Objective:

Verify that the jump concatenation logic generates the correct jump target address and updates the program counter when a jump instruction is executed.

Signals Observed:

- Jump
- jump_value (hex)
- target28 (hex)

Expected Behavior:

target28	jump_value
00000004	00000004

Observed Behavior:

During the jump instruction, the target address is correctly generated and points to address 00000004, represented by jump_value.

The program counter is updated with this value only when the Jump control signal is high. Although this test jumps to address 00000004, other jump addresses can also be selected depending on the instruction target field.

Conclusion:

The jump concatenation module works correctly. In instructions.mem, the jump instruction was set to target address 00000004, and the waveform confirms that jump_value matches the expected result. No bugs were observed in this hardware block.

- PC Register

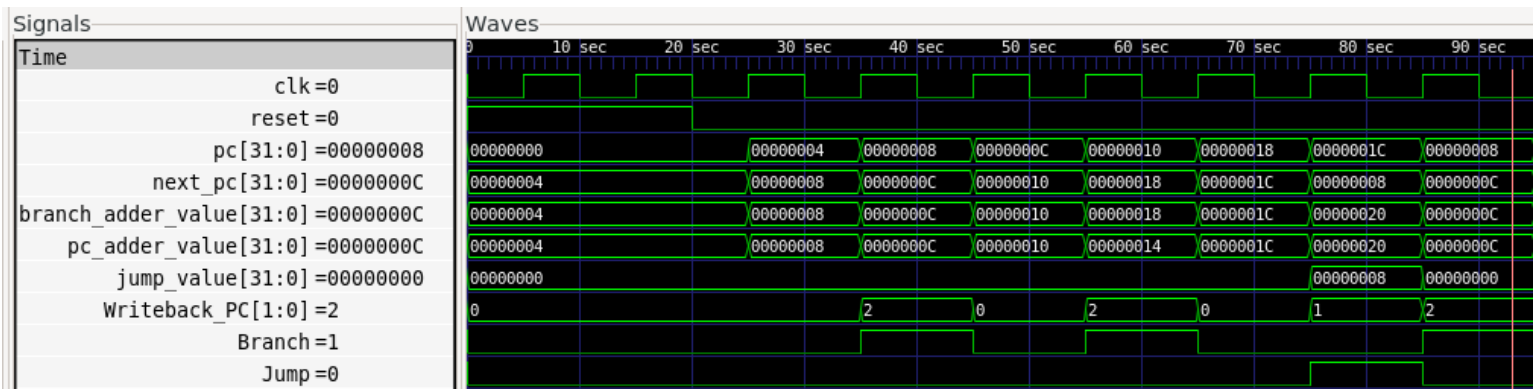


Image 7

Test Objective:

Verify that the PC register selects the correct next program counter address, next_pc, from three possible inputs, pc_adder_value, branch_adder_value, jump_value. All three PC update paths are tested in the same simulation.

Signals Observed:

- next_pc (hex)
- pc_adder_value (hex)
- branch_adder_value (hex)
- jump_value (hex)
- Writeback_PC (signed decimal)

Expected Behavior:

Writeback_PC	pc_adder_value	next_pc	At time
0	00000008	00000008	30 sec

Writeback_PC	pc_adder_value	branch_adder_value	next_pc	At time
2	00000014	00000018	00000018	60 sec

Writeback_PC	pc_adder_value	jump_value	next_pc	At time
1	00000020	00000008	00000008	80 sec

Observed Behavior:

This test combines the previously verified PC flow components. The next_pc signal is selected from one of three possible inputs using a MUX controlled by the Writeback_PC signal.

When Writeback_PC = 0, the PC register selects pc_adder_value, allowing the processor to continue to the next sequential instruction.

When `Writeback_PC = 2`, the PC register selects `branch_adder_value`, allowing the processor to branch to the calculated branch target.

When `Writeback_PC = 1`, the PC register selects `jump_value`, allowing the processor to jump to the target address.

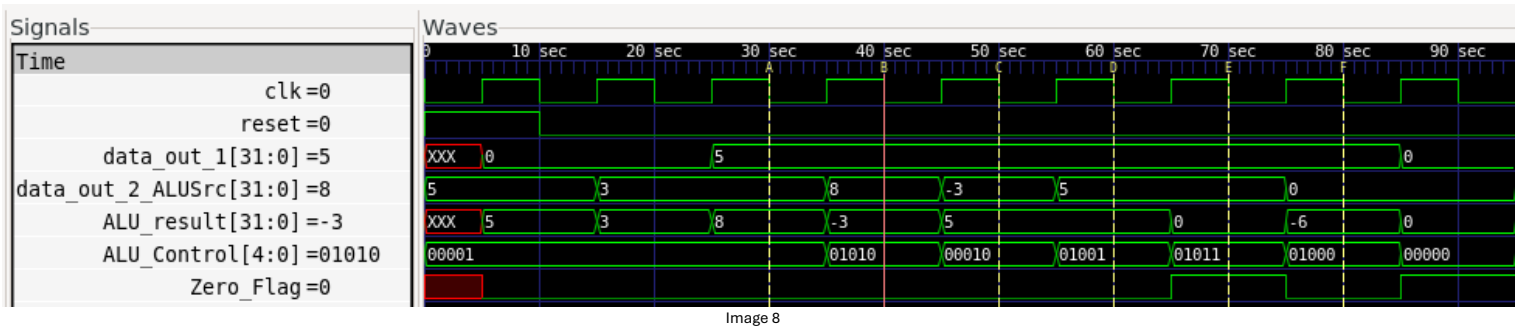
Branch and jump instructions do not occur at the same time in this test, so each PC update path can be verified independently.

Conclusion:

The PC register correctly selects the next program counter value based on the executed instruction and the `Writeback_PC` control signal. All three paths, sequential PC update, branch target, and jump target, behave as expected.

2. Arithmetic Units

- ALU



Test Objective:

Verify that the ALU performs the correct arithmetic and logic operations on two register values. The two registers are first initialized using ADDI instructions at 10 sec and 20 sec.

Signals Observed:

- data_out_1 (signed decimal)
- data_out_2_ALUSrc (signed decimal)
- ALU_result (signed decimal)
- ALU_Control (binary)

Expected Behavior:

At time	30 sec	40 sec	50 sec	60 sec	70 sec	80 sec
data_out_1	5 (0101)	5	5	5	5	5
data_out_2_ALUSrc	3 (0011)	3	3	3	3	3
ALU_Control	00001 (ALU_ADD)	01010 (ALU_SUB)	00010 (ALU_AND)	01001 (ALU_OR)	01011 (ALU_XOR)	01000 (ALU_NOR)
ALU_result	8	2	1	7	6	-8

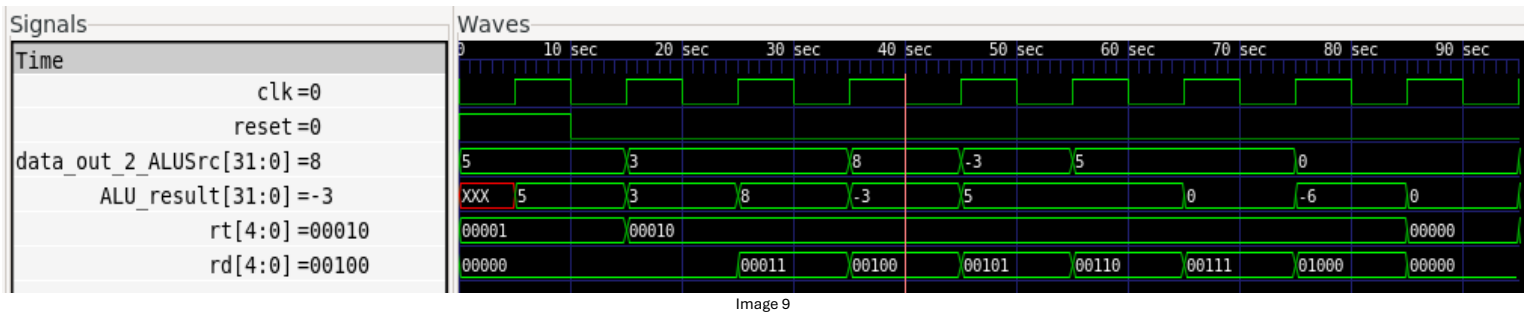
Observed Behavior:

A result mismatch is observed from 40 sec to 80 sec when comparing data_out_2_ALUSrc and ALU_result with the expected behavior.

The waveform shows a pattern: data_out_1, which comes from register 1, remains stable. However, data_out_2_ALUSrc, which comes from register 2, changes based on the ALU_result from the previous cycle.

The test instructions were designed so that register 1 and register 2 should not be overwritten. Since register 2 is changing unexpectedly, this suggests a design issue.

Root Cause:



In the observed behavior, data_out_2_ALUSrc, which comes from register 2, is being overwritten. Register 2 has address 00010 in the register file.

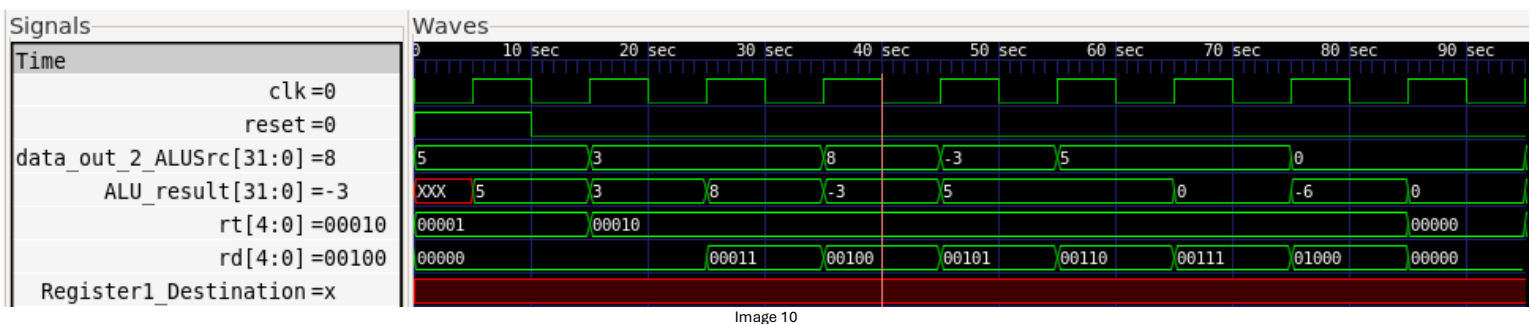
The processor uses two instruction fields for destination register selection:

rt[4:0] → destination for I-type instructions

rd[4:0] → destination for R-type instructions

For ALU R-type instructions, the destination register should be selected using rd[4:0]. However, the waveform shows that rt[4:0] is being used instead. This causes register 2 to be overwritten, even though it should only be used as a source register.

The destination register is selected using a MUX controlled by the Register1_Destination signal inside the Register_Writeback_1 module.



After expanding the waveform signals, Register1_Destination is shown as undefined for the entire instruction sequence. Because this control signal is undefined, the register file does not receive the correct destination address, causing register 2 to be overwritten each cycle.

Debugging Operation:

```
// CONTROL UNIT
Control_Unit control(
    .op(op),
    .fn(fn),
    .ImmExt(ImmExt),
    .ALU_Control(ALU_Control),
    .ALUSrc(ALUSrc),
    .Register1_Write(Register1_Write),
    .HILO_Select(HILO_Select),
    .Writeback_Control(Writeback_Control),
    .Memory1_Write(Memory1_Write),
    .Jump(Jump),
    .Branch(Branch),
    .Writeback_PC(Writeback_PC)
);
```

```
Control_Unit.sv
1 module Control_Unit (
2     input opcode_t op,
3     input funct_t fn,
4     output logic ImmExt,
5     output alu_ctrl_t ALU_Control,
6     output logic ALUSrc,
7     output logic Register1_Write,
8     output HILO_select_t HILO_Select,
9     output writeback_t Writeback_Control,
10    output logic Memory1_Write,
11    output logic Jump,
12    output logic Branch,
13    output writeback_pc_t Writeback_PC,
14    output logic Register1_Destination,
15    output logic HILO_Register_Enable
16 );
```

Starting from the CPU top module, MIPS.sv, the two most relevant blocks are REGISTER WRITEBACK and CONTROL UNIT.

These instantiated blocks were compared with the main modules Control_Unit.sv and Register_Writeback_1.sv.

The issue was found in the control unit instantiation inside MIPS.sv. The following signals were missing, Register1_Destination and HILO_Register_Enable.

Because Register1_Destination was not connected, the writeback destination MUX could not correctly choose between rt and rd.

After updating the CONTROL UNIT block in MIPS.sv, the missing signals were properly connected.

```
// CONTROL UNIT
Control_Unit control(
    .op(op),
    .fn(fn),
    .ImmExt(ImmExt),
    .ALU_Control(ALU_Control),
    .ALUSrc(ALUSrc),
    .Register1_Write(Register1_Write),
    .HILO_Select(HILO_Select),
    .Writeback_Control(Writeback_Control),
    .Memory1_Write(Memory1_Write),
    .Jump(Jump),
    .Branch(Branch),
    .Writeback_PC(Writeback_PC),
    .Register1_Destination(Register1_Destination),
    .HILO_Register_Enable(HILO_Register_Enable)
);
```

New observed Behavior:

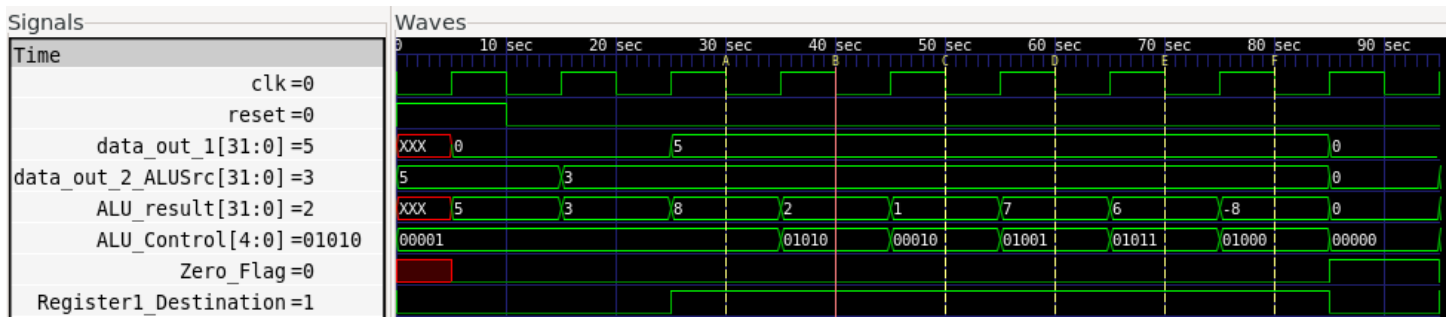


Image 11

After the fix, the waveform matches the expected behavior at each tested time.

data_out_1 and data_out_2_ALUSrc remain stable and are no longer overwritten. The ALU operations match the selected ALU_Control values, and the ALU_result values are correct.

The Register1_Destination signal is now 1 for R-type instructions, which means the processor correctly selects rd[4:0] instead of rt[4:0].

Conclusion:

The ALU itself behaved correctly. The issue was caused by a missing instantiation connection in the CPU top module, which left Register1_Destination undefined.

After connecting the missing control signals, the ALU operations matched the expected results, and no further bugs were found in this hardware block.

- Multiplier

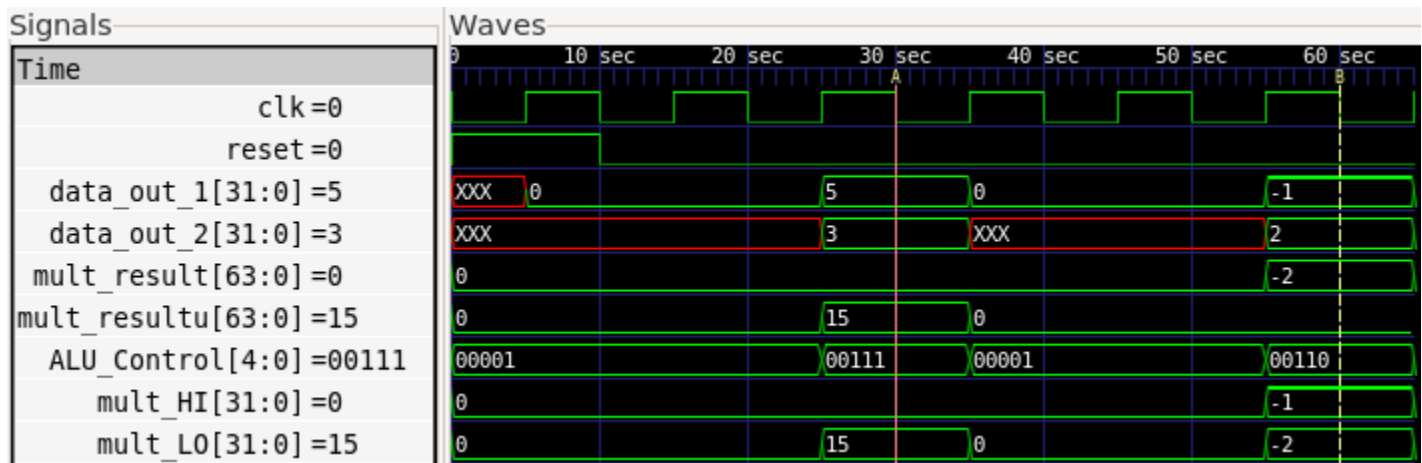


Image 12

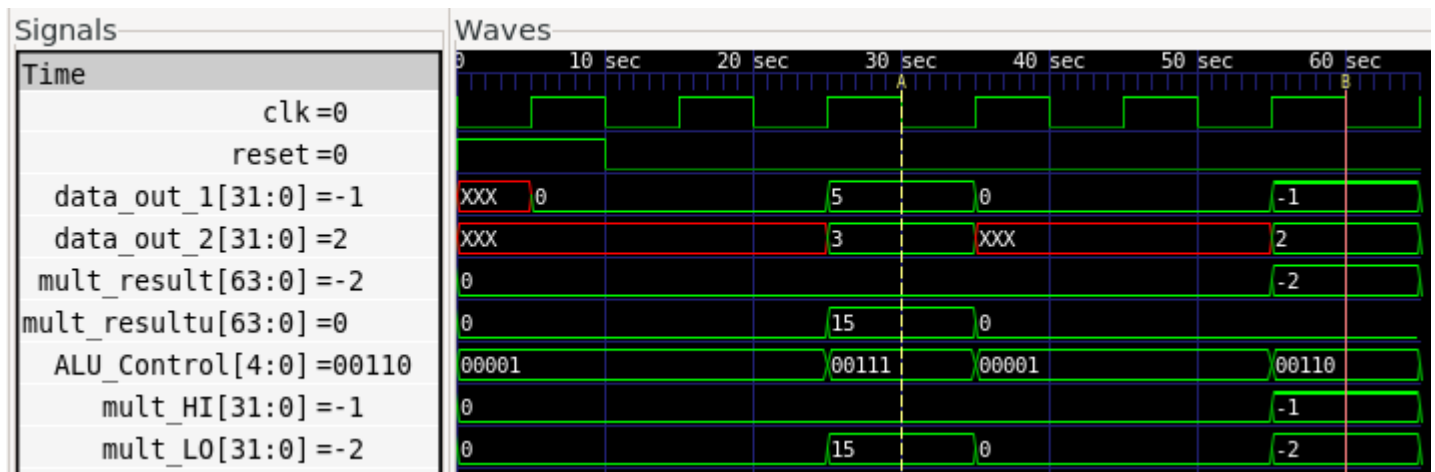


Image 13

Test Objective:

Verify that the multiplier unit correctly performs both signed and unsigned multiplication using two register operands.

Signals Observed:

- data_out_1(signed decimal)
- data_out_2 (signed decimal)
- mult_result (signed decimal)
- mult_resultu (decimal)
- ALU_Control (binary)

Expected Behavior:

Unsigned multiplication (image 12)

data_out_1	data_out_2	ALU_Control (ALU_MULTU)	mult_resultu
5	3	00111	15

Signed multiplication (image 13)

data_out_1	data_out_2	ALU_Control (ALU_MULT)	mult_result
-1	2	00110	-2

Observed Behavior:

The ALU_Control signal correctly selects between signed and unsigned multiplication modes.

For unsigned multiplication, data_out_1 = 5 and data_out_2 = 3, producing:

mult_resultu = 15

For signed multiplication, data_out_1 = -1 and data_out_2 = 2, producing:

mult_result = -2

Both results match the expected behavior.

The signals mult_HI and mult_LO represent the upper 32 bits and lower 32 bits of the multiplication result. These signals are connected to the HI/LO register path and are covered in more detail in the HI/LO register verification section.

Some undefined values appear on data_out_2 during instruction transitions. These are caused by the setup instructions and ALU input switching before the multiplication instruction is active. Since the final multiplication results are correct, these temporary undefined values do not indicate a multiplier issue.

Conclusion:

The multiplier unit operates correctly for both signed and unsigned multiplication. The control unit sends the correct ALU_Control values, and the multiplier outputs match the expected results. No issues were observed in this hardware block.

- Divider

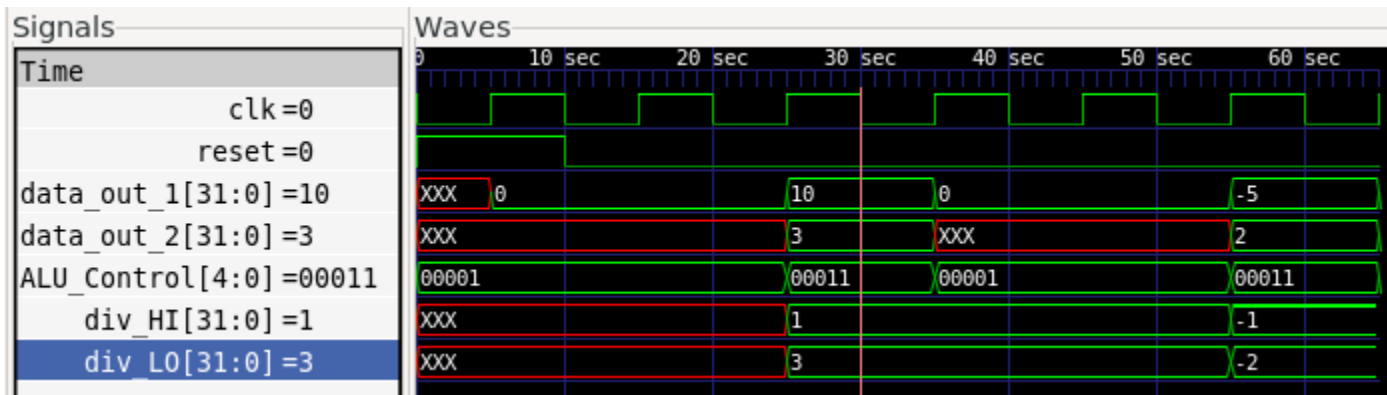


Image 14

Test Objective:

Verify that the divider unit correctly divides two register values and outputs both the quotient and the remainder.

Signals Observed:

- data_out_1 (signed decimal)
- data_out_2 (signed decimal)
- ALU_Control (binary)
- div_HI (signed decimal)
- div_LO (signed decimal)

Expected Behavior:

At time	ALU_Control (ALU_DIV)	data_out_1	data_out_2	div_LO (Quotient)	div_HI (Remainder)
30 sec	00011	10	3	3	1

Observed Behavior:

The divider correctly divides the two register values and produces two outputs, div_LO = quotient, div_HI = remainder.

At 30 sec, the divider performs $10 / 3 = 3$ remainder 1.

The waveform shows that div_LO = 3 and div_HI = 1, which matches the expected behavior.

The ALU_Control signal correctly selects the divide operation. At 60 sec, a second divide operation is performed using signed values, and the outputs also match the expected signed division behavior.

Division by zero is considered undefined behavior and is not tested as a valid operation in this verification.

Conclusion:

The divider unit correctly outputs the quotient and remainder for the tested divide instructions. The control signal selects the correct operation, and the generated values match the expected behavior. No issues were found in this hardware block.

3. Register and Memory

- HI/LO Register

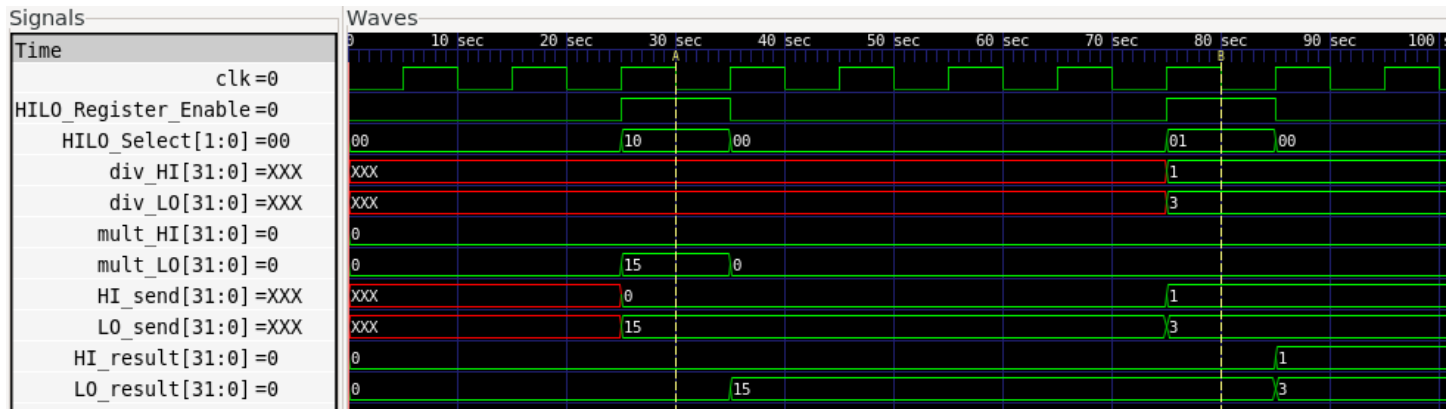


Image 15

Test Objective:

Verify HI/LO register correctly selects, stores, and outputs HI and LO values based on multiplication and division operations.

Signals Observed:

- HILO_Register_Enable
- HILO_Select (binary)
- HI_result (signed decimal)
- LO_result (signed decimal)

Expected Behavior:

Multiplication

Time	HILO_Select	HI_send	LO_send
30 sec	10	0	15

Division

Time	HILO_Select	HI_send	LO_send
80 sec	01	1	3

Observed Behavior:

At both tested times, the control unit selects the correct HILO_Select value. This signal is used by the HI/LO multiplexers to choose between multiplication and division outputs.

Since the processor has one HI/LO register pair, the HI/LO path must select between mult_HI / mult_LO OR div_HI / div_LO.

The selected values are output as:

HI_send

LO_send

For multiplication, the HI/LO path correctly selects:

HI_send = 0

LO_send = 15

For division, the HI/LO path correctly selects:

HI_send = 1

LO_send = 3

These values are then stored in the HI and LO registers as HI_result and LO_result.

Some undefined values appear earlier in the waveform because multiplication and division instructions have not yet executed, so the corresponding hardware outputs are still uninitialized. This does not indicate an issue because the final selected and stored values match the expected behavior.

Conclusion:

The HI/LO register path works correctly. The control unit selects the correct multiplication or division result, and the HI and LO values are stored properly. The stored results are then available for the register writeback MUX.

- Register File

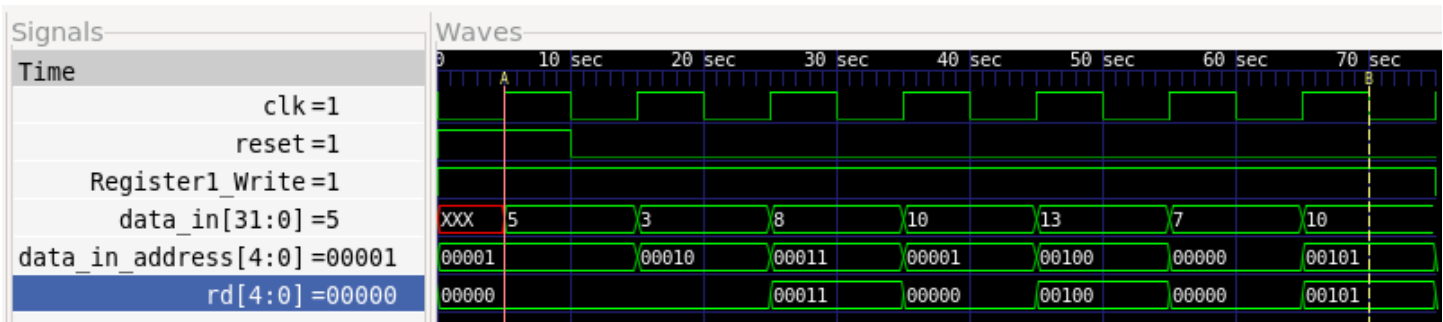


Image 16

Test Objective:

Verify that the register file correctly writes data to the selected destination register and that register \$0 remains hardwired to zero.

Signals Observed:

- data_in (signed decimal)
- data_in_address (binary)
- rd (binary)

Expected Behavior:

Time at	rd	data_in_address	data_in
30 sec	00011	00011	8
50 sec	00100	00100	13
70 sec	00101	00101	3

Observed Behavior:

At 30 sec and 50 sec, the correct values are stored in the correct destination registers. However, at 70 sec, the wrong value is stored. The observed value is 10, while the expected value is 3.

This suggests that register \$0 is not behaving correctly. In MIPS, register \$0 must always remain equal to zero.

Root Cause:

At 70 sec, the test uses an instruction that adds register \$2 with register \$0.

The instruction sequence used to test register \$0 is:

20020003 → addi \$2, \$0, 3 // R2 = 3

20000007 → addi \$0, \$0, 7 // should not change R0

00022820 → add \$5, \$0, \$2 // R5 = 3

The expected result is:

$R0 = 0$

$R2 = 3$

$R5 = 3$

However, the waveform shows $\text{data_in} = 10$ at 70 sec. This means the second instruction successfully overwrote register \$0 with the value 7. Then the third instruction added:

$R0 + R2 = 7 + 3 = 10$

This confirms that register \$0 was being overwritten, which violates the MIPS register file rule.

To confirm this issue, temporary outputs were created for individual registers. In Image 17, temp0 represents register \$0, and temp2 represents register \$2. The waveform shows that temp0 is overwritten with 7, even though register \$0 should always remain zero.

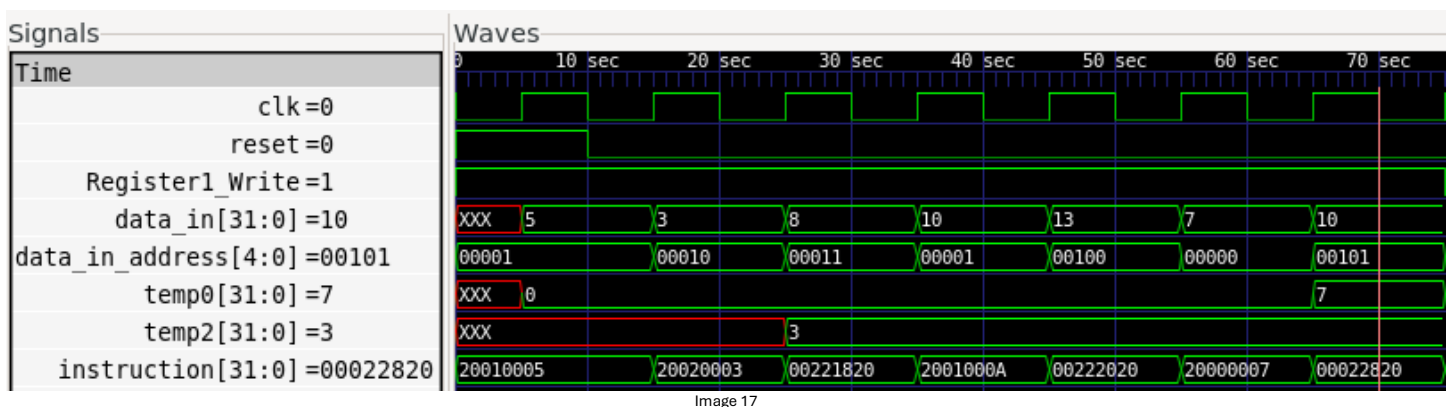


Image 17

A second issue was also observed during reset from 0 sec to 10 sec. While reset is high, the register values should be initialized to zero. However, the waveform shows undefined register values during this time. Unlike earlier undefined values caused by inactive instructions, this is a real issue because the register file should reset predictably before execution begins.

Debugging Operation:

To fix the first issue, the register file RTL was updated so that writeback only occurs when `Register1_Write` is high and `data_in_address` is not equal to zero.

Before the fix, the register file allowed writes to any address:

```

always_ff @(posedge clk or posedge reset) begin
    register1[0] <= 32'b0;
    if (reset) begin
        for (i = 0; i < 32 ;i = i + 1 ) begin
            register1[i] <= 32'b0;
        end
    end
    else if (Register1_Write) begin
        register1[data_in_address] <= data_in;
    end
end
end

```

After the fix, writes to register \$0 are blocked:

```

always_ff @(posedge clk or posedge reset) begin
    register1[0] <= 32'b0;
    if (reset) begin
        for (i = 0; i < 32 ;i = i + 1 ) begin
            register1[i] <= 32'b0;
        end
    end
    else if ((Register1_Write) && (data_in_address != 0)) begin
        register1[data_in_address] <= data_in;
    end
end
end

```

This prevents register \$0 from being overwritten.

For the reset issue, the register file module itself was not the main problem. After checking the CPU top module, MIPS.sv, the reset signal was found to be missing from the Register_1 module instantiation.

After connecting the reset signal to the register file instance, the register file reset behavior was corrected.

```

// REGISTER FILE
Register_1 reg1_file(
    .clk(clk),
    .Register1_Write(Register1_Write),
    .rs(rs),
    .rt(rt),
    .data_in(data_in),
    .data_in_address(data_in_address),
    .data_out_1(data_out_1),
    .data_out_2(data_out_2),
    .temp0(temp0),
    .temp2(temp2),
    .temp5(temp5)
);

```

After:

```
// REGISTER FILE
Register_1 reg1_file(
    .clk(clk),
    .reset(reset), // New update
    .Register1_Write(Register1_Write),
    .rs(rs),
    .rt(rt),
    .data_in(data_in),
    .data_in_address(data_in_address),
    .data_out_1(data_out_1),
    .data_out_2(data_out_2),
    .temp0(temp0),
    .temp2(temp2),
    .temp5(temp5)
);
```

New Observed Behavior:

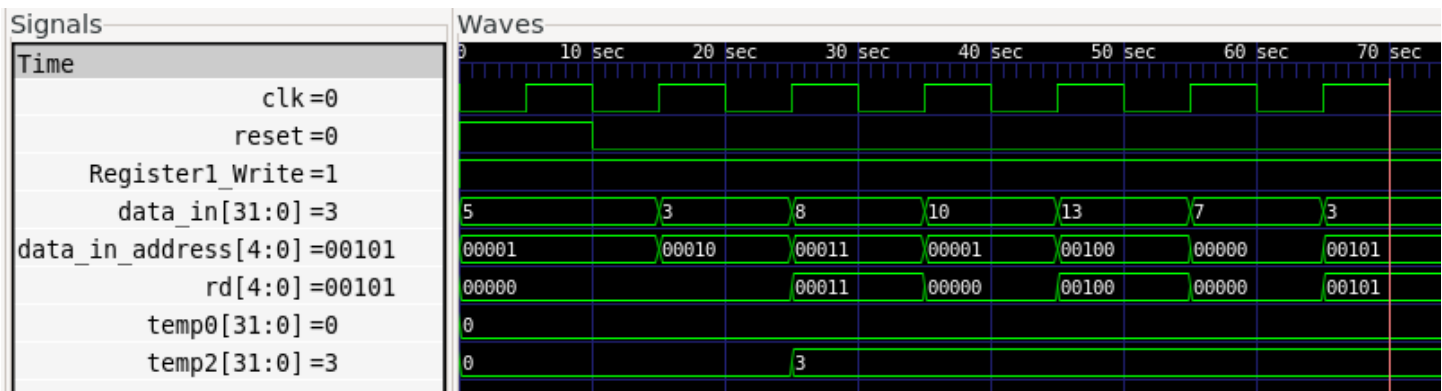


Image 18

At 70 sec, data_in now matches the expected value of 3. This confirms that register \$0 remains fixed at zero and is no longer overwritten.

The waveform also shows that during reset, the register values are correctly initialized to zero. This is shown using temporary register outputs such as temp0 and temp2.

Conclusion:

The register file had two issues. The first issue was inside the Register_1 module, where register \$0 could be overwritten. The second issue was in the CPU top module, where the reset signal was not connected to the register file instance.

After blocking writes to register \$0 and connecting the reset signal properly, the register file works correctly. No further issues were found.

- Data Memory

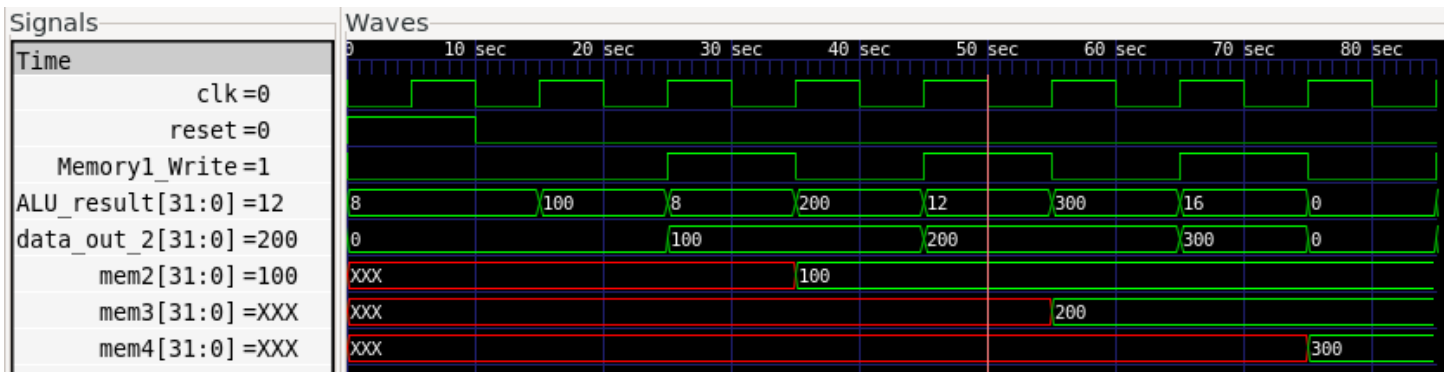


Image 19

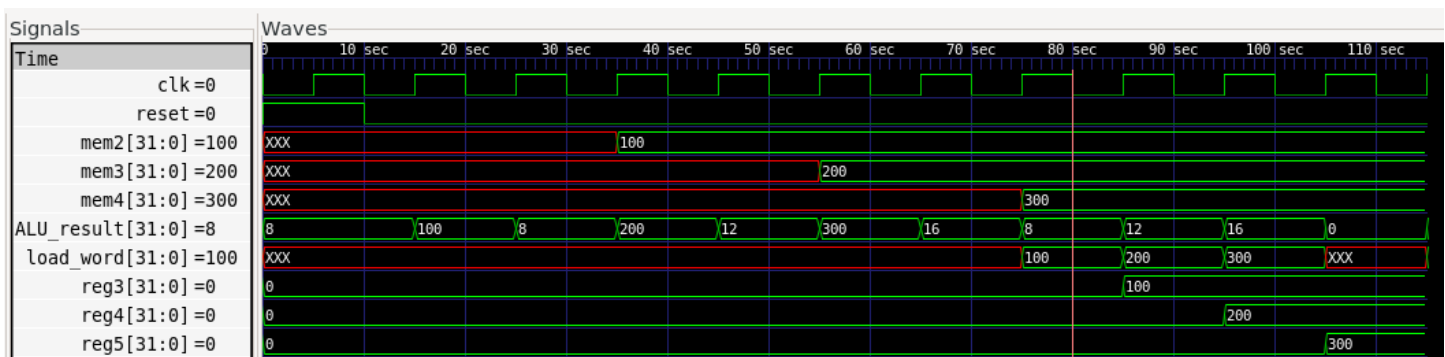


Image 20

Test Objective:

Verify that data can be correctly stored into memory and loaded back from memory.

Signals Observed:

- Memory1_Write
- ALU_result (signed decimal)
- data_out_2 (signed decimal)
- load_word (signed decimal)

Expected Behavior:

Store Word (image 19)

At time	ALU_result	data_out_2	Memory address to write
30 sec	8	100	2
50 sec	12	200	3
70 sec	16	300	4

Since this is a 32-bit design with byte addressing, the memory index is found by shifting the byte address right by 2 bits. Therefore, every time ALU_result increases by 4, the memory index increases by 1.

Load Word (image 20)

At time	ALU_result	load_word	Memory address to read
80 sec	8	100	2
90 sec	12	200	3
100 sec	16	300	4

Observed Behavior:

When Memory1_Write is high, the ALU correctly calculates the memory address by adding the base register and offset. The resulting address is then shifted to select the correct memory index.

On the rising clock edge, the value from data_out_2 is stored into the selected memory location. In Image 19, three temporary outputs are used to confirm that the correct values were stored:

```
assign mem2 = data_memory[2];
assign mem3 = data_memory[3];
assign mem4 = data_memory[4];
```

The waveform confirms:

mem2 = 100

mem3 = 200

mem4 = 300

For the load word test, the same addressing method is used to read values from memory. In Image 20, the values stored in memory are loaded back into the register file.

Temporary register outputs are used to confirm the loaded values:

```
assign reg3 = register1[3];
assign reg4 = register1[4];
assign reg5 = register1[5];
```

The waveform confirms:

reg3 = 100

reg4 = 200

reg5 = 300

A one-cycle delay is observed between the memory read and the register update. This is expected because the loaded value must pass through the writeback path before being stored in the register file.

Conclusion:

The data memory correctly stores values at the selected memory addresses and loads them back successfully. Addressing works as expected using byte-addressed offsets, where the memory index increases by 1 for every 4-byte increase in ALU_result.

The observed one-cycle delay between memory read and register update confirms correct sequential writeback behavior.

4. Control Logic

- Instruction Decoder

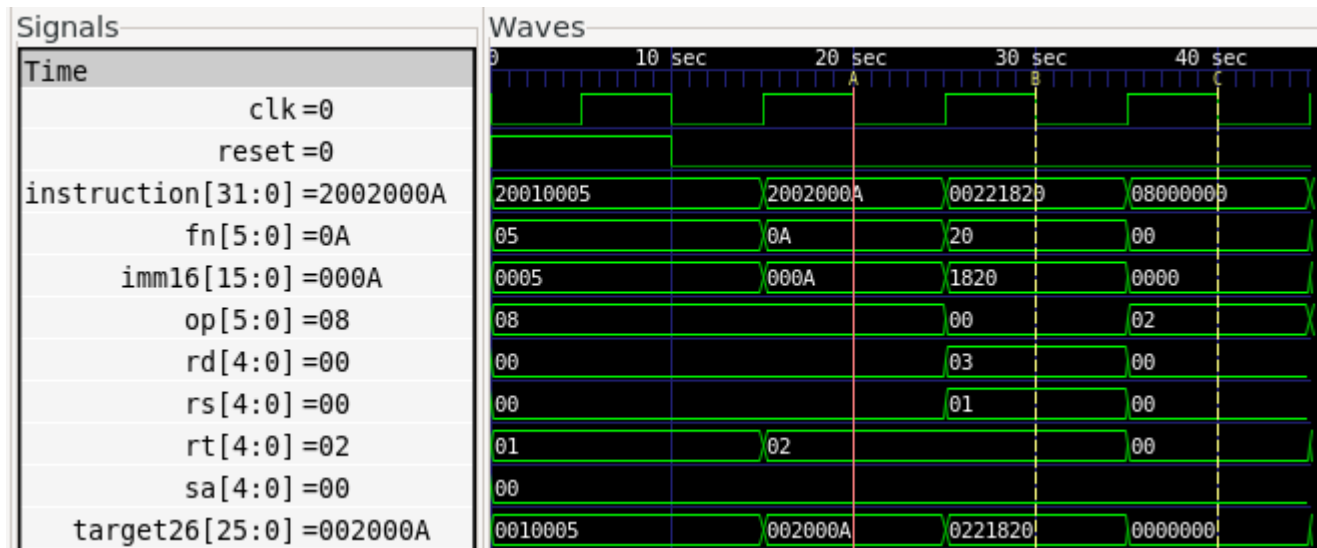


Image 21

Test Objective:

Verify that the instruction decoder correctly extracts instruction fields for all three supported instruction types: R-type, I-type, J-type

Signals Observed:

- Instruction (hex)
- fn (function) (hex)
- op (opcode) (hex)
- rd (register destination) (hex)
- rs (first register source) (hex)
- rt (second register source OR register target) (hex)
- imm16 (16-bit immediate) (hex)
- target26 (26-bit target) (hex)

Expected Behavior:

I Type

At time	Instruction	op	rt	rs	Imm16
20 sec	2002000A	08	02	00	000A

R Type

At time	Instruction	op	fn	rt	rs	rd
30 sec	00221820	00	20	02	01	03

J Type

At time	Instruction	op	target26
40 sec	08000000	02	0000000

Observed Behavior:

For an R-type instruction, the instruction bits are split into six fields:

Fields	op	rs	rt	rd	sa	fn
Bits	31-26	25-21	20-16	15-11	10-6	5-0

For an I-type instruction, the instruction bits are split into four fields:

Fields	Op	rs	rt	imm16
Bits	31-26	25-21	20-16	15-0

For a J-type instruction, the instruction bits are split into two fields:

Fields	op	target26
Bits	31-26	25-0

The I-type instruction tested is:

2002000A

This represents an ADDI instruction. The opcode is 08, which selects the add-immediate operation. The instruction adds the immediate value 000A to the value in register rs, which is register \$0, and stores the result in register rt, which is register \$2.

The R-type instruction tested is:

00221820

For R-type instructions, the opcode is always 00, and the fn field determines the operation. In this case, fn = 20, which represents an ADD operation. The instruction adds the values from rs = 01 and rt = 02, then stores the result in rd = 03.

The J-type instruction tested is:

08000000

For J-type instructions, the opcode 02 represents a jump instruction. The target address is extracted from target26. In this test, target26 = 0000000, which makes the program counter jump to the selected target address.

In all three instruction types, the decoder correctly slices the instruction into the required fields. The op field is used by the control unit to generate datapath control signals, while the fn field is used only for R-type instructions to identify the specific ALU operation.

Conclusion:

The instruction decoder correctly extracts the opcode, register fields, immediate value, function field, and jump target field from R-type, I-type, and J-type instructions.

Since the decoder uses fixed bit slicing and does not depend on the instruction behavior itself, verifying one instruction from each instruction format is enough to confirm that the decoder is working correctly. No issues were observed in this hardware block.

- Control Unit

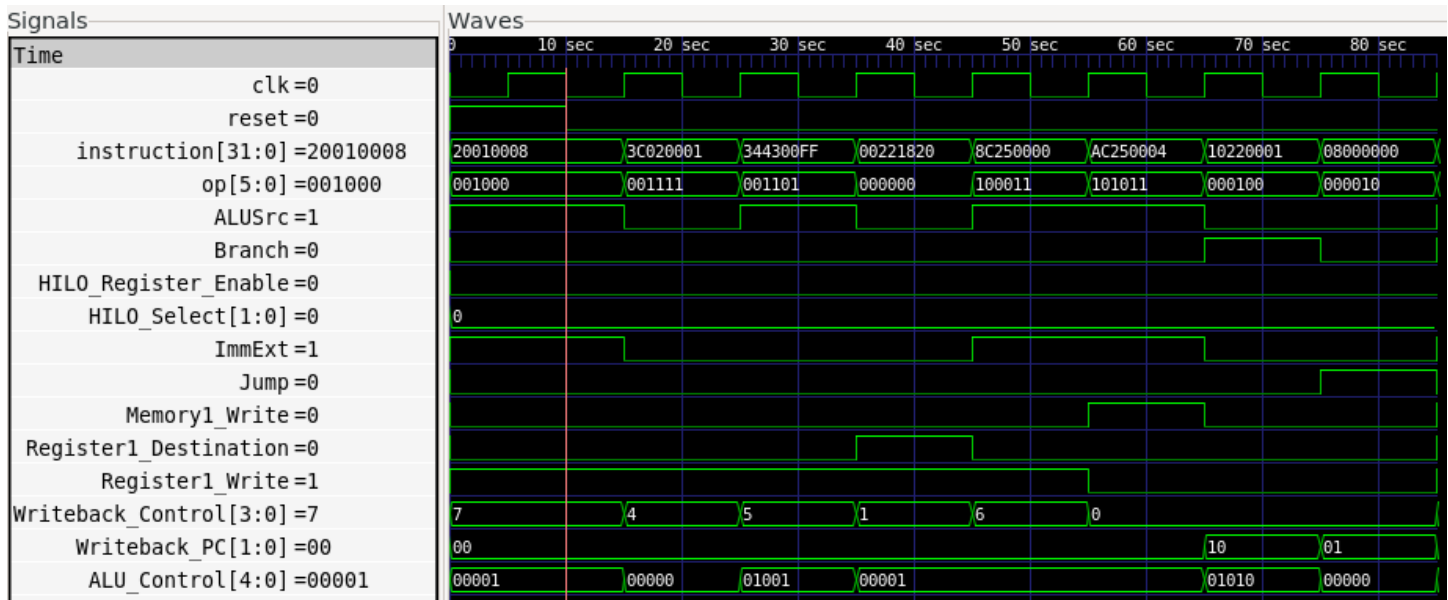


Image 22

Test Objective:

Verify that the control unit generates the correct control signals for each supported instruction opcode.

The tested opcodes are:

R Type (ALU)	Load Word (LW)	Store Word (SW)	Branch equal (BEQ)	Load upper immediate (LUI)	OR immediate (ORI)	Add Immediate (ADDI)	Jump (J)
000000	100011	101011	000100	001111	001101	001000	000010

Signals Observed:

- op (opcode) (hex)
- ALUSrc (ALU source)
- Branch
- ImmExt (immediate extend)
- Jump
- Memory1_Write
- Register1_Destination
- Register1_Write

Expected Behavior:

At time	10 sec	20 sec	30 sec	40 sec	50 sec	60 sec	70 sec	80 sec
op	001000	001111	001101	000000	100011	101011	000100	000010
ALUSrc	1	0	1	0	1	1	0	0
Branch	0	0	0	0	0	0	1	0
ImmExt	1	0	1	0	1	1	0	0
Jump	0	0	0	0	0	0	0	1
Memory1_Write	0	0	0	0	0	1	0	0
Register1_Destination	0	0	0	1	0	0	0	0
Register1_Write	1	1	1	1	1	0	0	0

Observed Behavior:

The control unit generates the expected control signals for each opcode.

Some control signals, such as ALU_Control, are not included in the expected behavior table because they were already tested in earlier verification sections. This section focuses on the main datapath control signals generated directly from the opcode.

Each control signal has a default value of 0. At the beginning of each control decision, the control unit sets signals to their default values and then changes only the required signals based on the opcode. This prevents unintended latch behavior and keeps unused control paths disabled.

For example:

- SW enables Memory1_Write but disables register writeback.
- BEQ enables the Branch signal.
- J enables the Jump signal.

The waveform confirms that the correct signals are activated for each tested instruction type

Conclusion:

The control unit correctly generates opcode-based control signals for all tested instruction types. Since previous verification sections confirmed that the datapath responds correctly to these control signals, this test validates the opcode-to-control-signal mapping. No issues were observed in this module.

5. Writeback Path

- Register Writeback 1

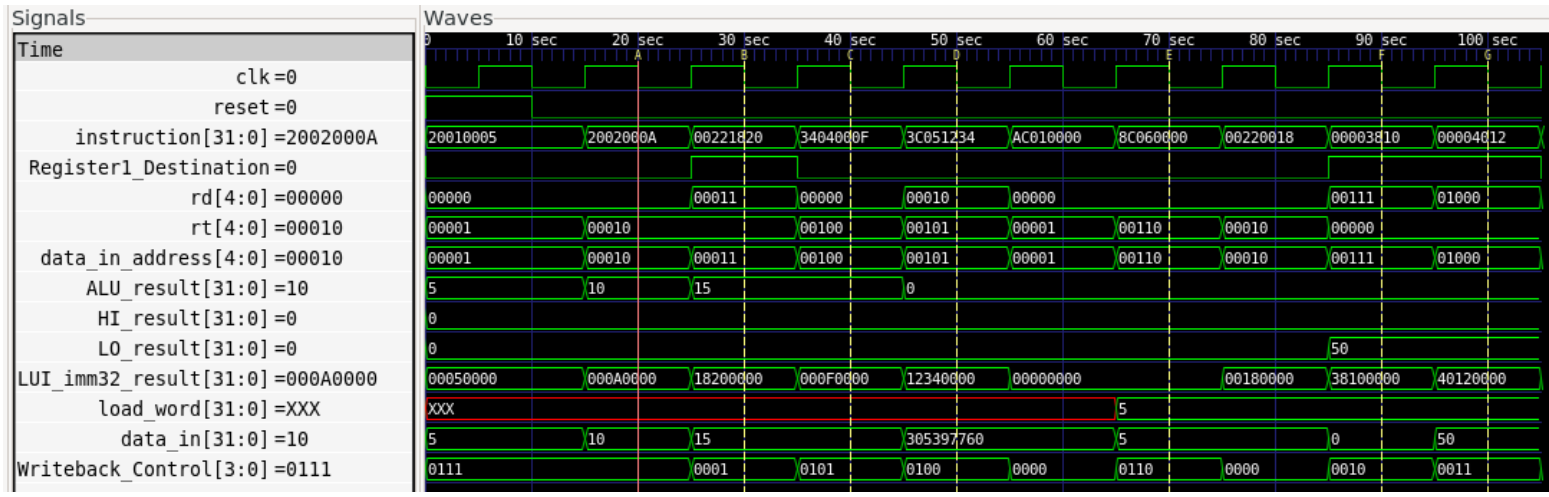


Image 23

Test Objective:

Verify that the register writeback module selects the correct destination address and data value for the register file.

The destination address is selected using Register1_Destination, and the writeback data is selected using Writeback_Control.

Signals Observed:

- Register1_Destination
- data_in_address (binary)
- data_in (signed decimal)
- Writeback_Control (binary)

Expected Behavior:

At time	Instruction	Register1_Destination	data_in_address	data_in	Writeback_Control
20 sec	ADDI	0	00010	10	0111
30 sec	ADD	1	00011	15	0001
40 sec	ORI	0	00100	15	0101
50 sec	LUI	0	00101	0x1234 (hex)	0100
70 sec	LW	0	00110	5	0110
90 sec	MFHI	1	00111	0	0010
100 sec	MFLO	1	01000	50	0011

Observed Behavior:

Two control signals determine the final writeback address and writeback data, Register1_Destination and Writeback_Control.

Register1_Destination selects the destination register address. When the signal is 0, the module selects rt, which is used by I-type instructions such as ADDI, ORI, Lui, and LW. When the signal is 1, the module selects rd, which is used by R-type instructions such as ADD, MFHI, and MFLO.

The selected destination register is then copied into data_in_address, which is used as the register file write address.

Writeback_Control selects the value that will be written into the register file. This value gets copied into data_in register, which will be used by the register file.

The writeback selections are defined as:

Writeback	NONE	ALU	HI	LO	LUI	ORI	LW	ADDI
Binary	0000	0001	0010	0011	0100	0101	0110	0111

This table matches the definitions used in MIPS_Definitions.sv and the control logic in CONTROL UNIT. Since the observed Writeback_Control values match the expected instruction behavior, the control unit is confirmed to be producing the correct writeback selection signals.

The waveform also shows that load_word is undefined from 0 sec to about 65 sec. This is expected because memory has not been written or read yet. Since load_word is only selected during LW, this undefined value does not affect earlier instructions.

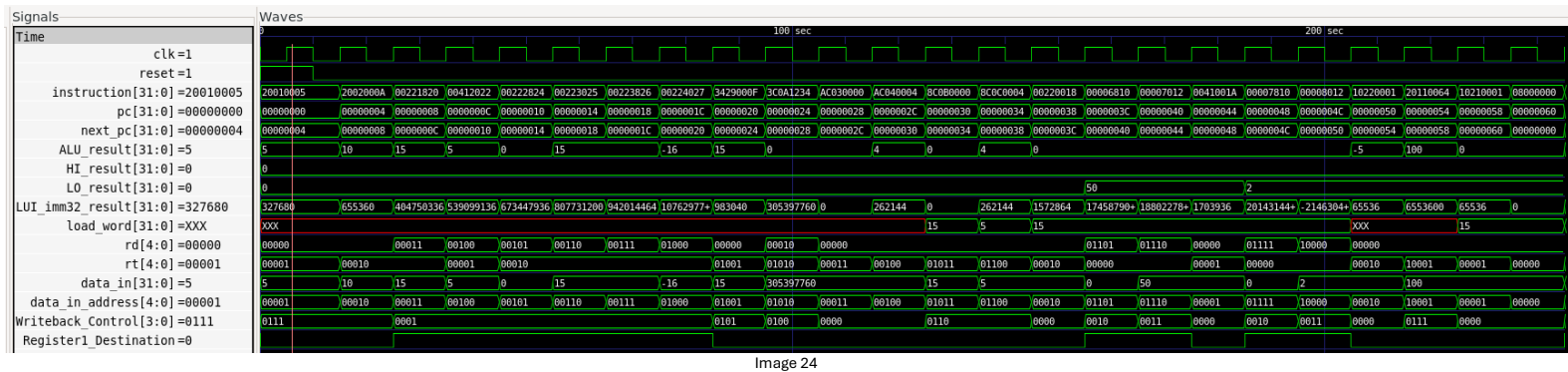
Conclusion:

The writeback module correctly selects both the register destination address and the data value written into the register file.

Register1_Destination correctly selects between rt and rd, while Writeback_Control correctly selects between ALU, HI, LO, Lui, ORI, LW, and ADDI writeback sources.

No issues were found in the writeback path.

- Final Overall Check



Test Objective:

Verify the complete single-cycle CPU at the system level to ensure that all previously verified hardware blocks work correctly together.

This test checks that:

- unintended latches are not created
- X propagation does not affect valid operations
- registers store the correct values
- memory stores the correct values at the correct addresses
- the full datapath behaves correctly during instruction execution

Observed Behavior:

Although each unit was verified individually in previous sections, the final overall check verifies the CPU as a complete system. To keep this verification simple and non-redundant, the focus is on checking key register and memory outputs rather than repeating every module-level test.

The main purpose of this final check is to confirm that no unintended latch behavior occurs before moving into the pipelining and FPGA implementation stages.

In the waveform, `load_word` shows temporary X propagation when memory has not yet been written or read. This is expected because the data memory is not initialized by reset in this design. Since `load_word` is only used during valid load instructions, these undefined values do not affect correct CPU operation.

The final check confirms that the important outputs are valid when they are needed. Register writeback, memory stores, memory loads, PC updates, and control signals all behave correctly during the full instruction sequence.

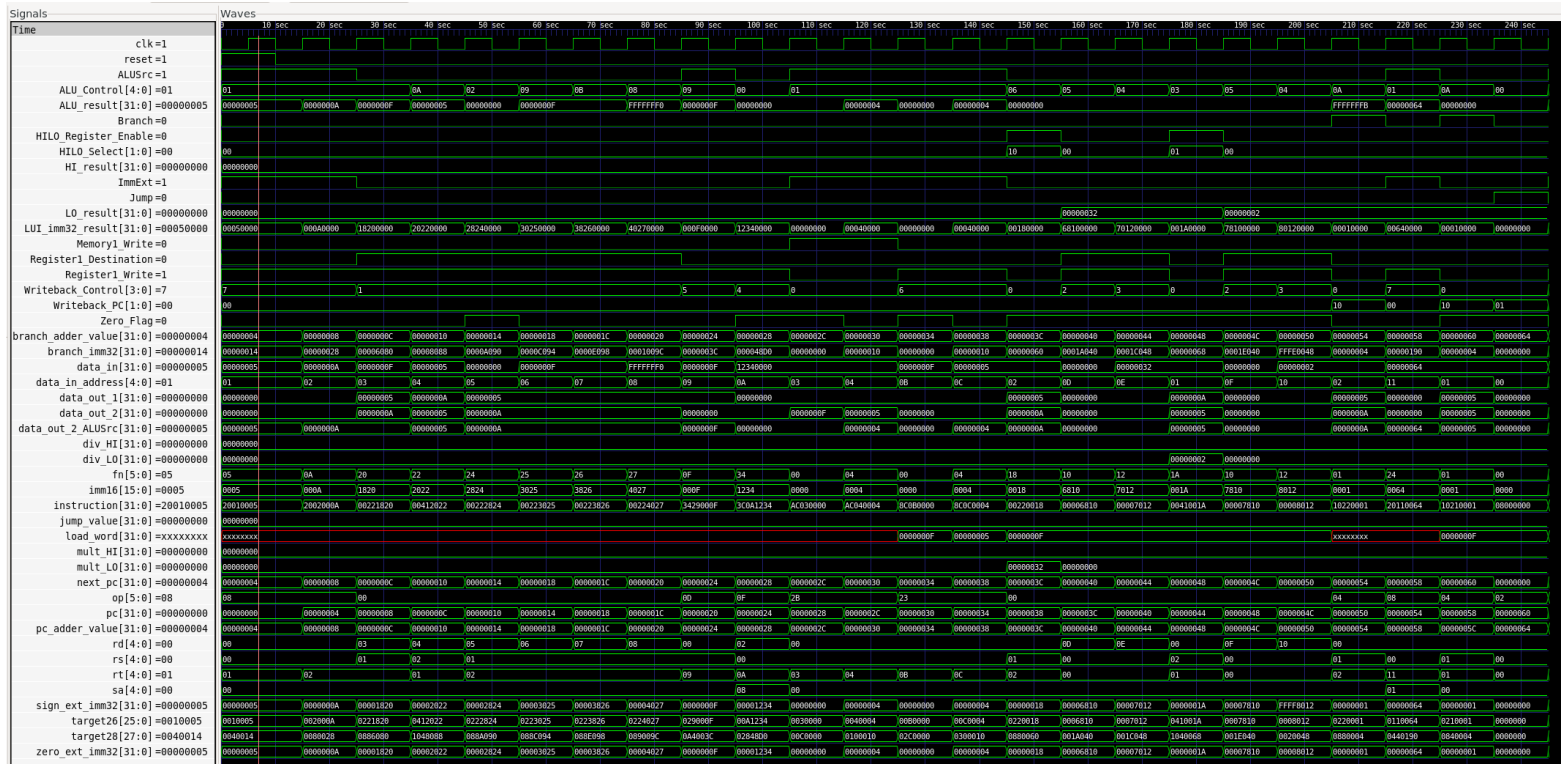


Image 25

Conclusion:

The complete single-cycle CPU operates correctly at the system level. The previously verified hardware blocks work together as expected, and no unintended latch behavior was observed in valid datapath operations.

The remaining temporary undefined values are limited to unused or uninitialized memory outputs and do not affect correct instruction execution. This confirms that the single-cycle CPU is ready to be used as the foundation for pipelining, hazard handling, and FPGA implementation.